

RAM vs. Word-RAM, and the Church–Turing Thesis

CS1200: Introduction to Algorithms and their Limitations

Lecture 10

Fall 2026

Harvard SEAS

Put away your devices

- SRE 1 (Word-RAM simulation): second half of today.
- Pset 4 out today.
- Reading: Hesterberg–Vadhan §6.4, §7.4, §8.1–8.4.

Recap: the Word-RAM

Recap: a Word-RAM program

A **Word-RAM program** is a RAM program with a **word length** w (fixed for the run) and a **memory size** S (starts at n ; MALLOC adds one cell). Values live in $\{0, \dots, 2^w - 1\}$; $+$ and $\times$ saturate at $2^w - 1$. $P[w](x)$ halts with an output, runs forever, or crashes.

Def. 8.2. P **solves** $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if for every $x \in \mathcal{I}$:

1. for every w , $P[w](x)$ halts or crashes;
2. for all sufficiently large w , $P[w](x)$ does not crash;
3. whenever $P[w](x)$ does not crash, its output is in $f(x)$ (or is $\perp$ if $f(x) = \emptyset$).

Def. 8.3. $T_P(x) = \max_{w \in \mathbb{N}} T_{P[w]}(x)$ (crashed runs count; ∞ if unbounded).

Check: three programs, one input

Consider a problem Π and an input x with $f(x) = \{(5)\}$: the only correct output on x is (5). Each cell below is the outcome of a program run at that w and its number of steps.

	$w = 2$	$w = 3$	$w \geq 4$
P	crash, 3	output (5), 12	output (5), 12
Q	crash, 3	output (4), 12	output (5), 12
R	runs forever	output (5), 12	output (5), 12

Which of these programs do *not* solve Π ? Why? What is $T(x)$?

P : no failure on this x (the crash at $w = 2$ is allowed) $T_P(x) = 12$

Q : does not solve Π : wrong output at $w = 3$ (cond. 3) $T_Q(x) = 12$

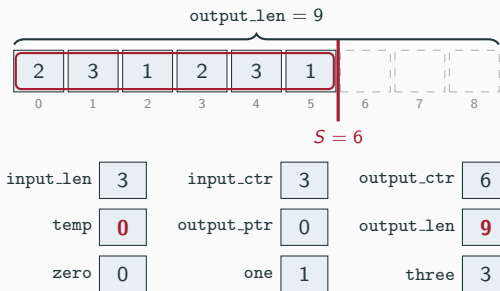
R : does not solve Π : runs forever at $w = 2$ (cond. 1) $T_R(x) = \infty$

A Word-RAM program

Duplicate

Input: an array x of length n . **Output:** the array of length $2n$: x followed by x . Here $x = (2, 3, 1)$, and w is large enough not to matter.

```
0 zero = 0;
1 one = 1;
2 three = 3;
3 input_ctr = 0;
4 output_ctr = input_len;
5   temp = input_len - input_ctr;
6   IF temp == 0 GOTO 13;
7   temp = M[input_ctr];
8   MALLOC;
9   M[output_ctr] = temp;
10  input_ctr = input_ctr + one;
11  output_ctr = output_ctr + one;
12  IF zero == 0 GOTO 5;
13 output_ptr = 0;
14 output_len = three × input_len;
```



Line 5: $temp = 0$, so line 6 jumps to 13. Lines 13–14 ask for 9 cells from $M[0]$; only $S = 6$ exist, so the output is cut at $M[S - 1]$: $(2, 3, 1, 2, 3, 1)$.

In groups: Duplicate at every word length

```
0 zero = 0;
1 one = 1;
2 three = 3;
3 input_ctr = 0;
4 output_ctr = input_len;
5   temp = input_len - input_ctr;
6   IF temp == 0 GOTO 13;
7   temp = M[input_ctr];
8   MALLOC;
9   M[output_ctr] = temp;
10  input_ctr = input_ctr + one;
11  output_ctr = output_ctr + one;
12  IF zero == 0 GOTO 5;
13 output_ptr = 0;
14 output_len = three × input_len;
```

1. For which word lengths w does $P[w](x)$ crash? At which step, and why?
 $w \leq 1$: at time 0 ($x[0] = 2 \geq 2^w$, and $3 \geq 2^w$)
 $w = 2$: at line 8, the 2nd MALLOC ($S = 5 > 4$)
2. How many commands does $P[w](x)$ execute for $w \geq 3$? For $w = 2$?
 $8n + 9 = 33$; 17 (counting the MALLOC that crashes)
3. $T_P(x)$ is the maximum of $T_{P[w]}(x)$ over all w . What is it? What is the worst case $T_P(n)$?
 33 : the crashed runs are shorter; $8n + 9 = \Theta(n)$

How big should W be?

Proposition 8.4

Recall (Def. 8.1): $P[w](x)$ crashes only if (1) a constant in P is $\geq 2^w$, (2) an input entry is $\geq 2^w$, or (3) $S > 2^w$.

Prop. 8.4. If $T_P(x) < \infty$, then $P[w](x)$ does not crash for any $w \geq w_0$, where

$$w_0 = \left\lceil \log_2 \max \left\{ \underbrace{n + T_P(x)}_{\text{bounds } S \text{ (3)}}, \underbrace{x[0], \dots, x[n-1]}_{\text{input entries (2)}}, \underbrace{c_0, \dots, c_{k-1}}_{\text{constants in } P \text{ (1)}} \right\} + 1 \right\rceil.$$

Why does $n + T_P(x)$ bound S ? S starts at n , and each step adds at most one cell.

Consequence: if $T_P(n) = \text{poly}(n)$ and inputs are at most $\text{poly}(n)$, then $w = O(\log n)$ suffices. (The constants are fixed: they don't grow with n .)

For a fixed machine, P is guaranteed to work on every input small enough that $w \geq w_0$. A 64-bit machine covers any realistic input.

Above w_0 , the **model** guarantees no crash; the **programmer** guarantees no wrong answer.

What does the Word-RAM model miss?

Word-RAM has been the dominant model in algorithms for over 50 years. Still:

- **Not all operations cost the same.** Usually they differ by constant or logarithmic factors.
- **The memory hierarchy.** Registers, cache, main memory, disk each have different access times and affordances.
- **Parallelism.** Working with p processors can speed up some tasks by a factor of p .

Last lecture's desiderata pull against each other; the Word-RAM balances simplicity, robustness, and technological relevance.

Representing data

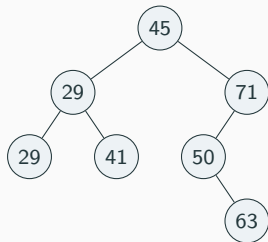
Everything is an array of natural numbers

Inputs and outputs of RAM programs are arrays of natural numbers. How do we represent:

signed integers	a natural number plus a separate sign bit
rationals	a pair of integers (numerator, denominator)
real numbers	impossible: the reals are uncountable; finite arrays of naturals are countable
strings	an array of ASCII codes in $\{0, \dots, 127\}$

Floating point ($x \cdot 2^y$) approximates reals, and almost every operation rounds. This course sticks to integers and rationals.

A binary search tree as an array



Each vertex is a 4-tuple (K, V, P_L, P_R) : key, value, and the *addresses* of its left and right children. 0 means “no child”.

Why can we use 0 to mean “no child”?

only the root is at address 0, and the root is not a child

K V P_L P_R

45	0	4	8	29	0	12	16	71	0	20	0	29	0	0	0	41	0	0	0	50	0	0	24	63	0	0	0
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27

Word-RAM vs. RAM

Runtimes of Word-RAM simulations of RAM programs

Thm. 8.5. Fix an encoding of inputs as arrays. Let $T(x) \geq n + 2$, with every input entry of bitlength $O(\log T(x))$. These are equivalent (either both are true or both are false):

1. Π is solved by a Word-RAM program P with $T_P(x) = O(T(x))$.
2. Π is solved by a RAM program Q with $T_Q(x) = O(T(x))$ such that, on every x ,
 - (a) every value in Q 's variables and memory has bitlength $O(\log T(x))$;
 - (b) the largest memory location Q uses (or outputs) is $O(T(x))$.

Q: What can a RAM do in one step that a Word-RAM can't do cheaply?

Cor. 8.6. A problem is solvable by a Word-RAM program if and only if it is solvable by a RAM program.*

*Without any efficiency guarantee: the Word-RAM program may be much slower.

Proof idea

Recall (Def. 8.2): P solves Π if, for every x : (1) on every w , $P[w](x)$ halts or crashes; (2) for all large enough w , it does not crash; (3) whenever it does not crash, its output is correct. Runtime is the max over all w , crashed runs included.

(2) $\Rightarrow$ (1): a Word-RAM program P imitates a non-cheating RAM program Q .

P does not know $T(x)$, so it guesses: $t = 1, 2, 4, 8, \dots$. For each guess t :

1. MALLOC about t cells: room for Q 's memory (uses (b)).
 - *Small machine*: it crashes here. Allowed by (2); it took $O(t)$ steps.
 - *Survives*: then $2^w \geq t$, so $w \geq \log_2 t$. Each of Q 's values fits in $O(1)$ words (uses (a)).
2. Simulate Q for t steps, $O(1)$ steps each. If Q finishes, output its answer: correct, so (3) holds. If not, double t .

Once $t \geq T(x)$ an attempt succeeds. Total: $1 + 2 + 4 + \dots \leq 4 T(x)$, so $O(T(x))$.

(1) $\Rightarrow$ (2): a RAM program imitates a Word-RAM program You solve this for PS4.

The Church–Turing Thesis

Thm. 7.5. If a problem is solvable in one of these models, it is solvable in all of them:

- RAM programs
- %-extended RAM programs
- Word-RAM programs
- Python, OCaml, C (with growing pointers)
- the λ -calculus (Church, 1936)
- Turing machines (Turing, 1936)
- ...

Such models are called **Turing-equivalent**.

This is a **theorem**: each pair is linked by a simulation.

The Church–Turing Thesis

Two variants:

1. The models of Thm 7.5 capture our intuitive notion of an algorithm.
2. A computation can occur in our universe if and only if it can be done in those models.

Why can't we prove it? “intuitive algorithm” and “physically possible” have no mathematical definition.

It has held up for 90 years — including quantum computers: anything a quantum algorithm can solve, a RAM program can solve, much more slowly.

The Extended Church–Turing Thesis

The Church–Turing Thesis is about *what* can be solved. The Extended Church–Turing Thesis is about *how fast*.

v1. Every physically realizable model of computation can be simulated by a Word-RAM program with only polynomial slowdown (and conversely).

Possible challenges: randomized algorithms, massively parallel computers, quantum computers

v2. Every physically realizable, **deterministic, and sequential** model of computation can be simulated by a Word-RAM program with only polynomial slowdown (and conversely).

Models polynomially equivalent to the Word-RAM (such as Turing machines) are **strongly Turing-equivalent**.

Theorem, thesis, or false?

1. Every Python program can be simulated by a RAM program. theorem (7.1)
2. A problem is solvable by a Word-RAM program iff it is solvable by a RAM program. theorem (Cor 8.6)
3. Anything a physical device can compute, a RAM program can compute. Church–Turing
4. A RAM program running in time T can be simulated by a Word-RAM program in time $O(T)$.
false
5. Every deterministic, sequential physical computer can be simulated by a Word-RAM program with polynomial slowdown. Extended CT, v2
6. Quantum computers can solve problems that no RAM program can solve. false

Takeaway

Takeaway

- The Word-RAM is the formal model under everything in this course.
- In practice: analyze the friendlier RAM model and check Theorem 8.5's conditions — no huge numbers, no huge addresses.
- Every algorithm so far passes that check, so their runtimes (InsertionSort n^2 , MergeSort $n \log n$, RadixSort, ...) are honest Word-RAM runtimes.
- From now on: high-level pseudocode. The models are the reference for what one step costs.

SRE 1

Sender–Receiver Exercise 1: a 2-D Word-RAM

Theorem: a 2-D Word-RAM program (memory is an $S \times S$ grid) can be simulated by an ordinary Word-RAM program, with runtime $O((n + t)^2)$.

- **Pair up:** one Sender, one Receiver. No partner? Raise your hand; a group of three is fine.
- **Sender:** explain the theorem and proof, top-down — the main idea first, then the details. Avoid writing; one diagram to get started is fine.
- **Receiver:** take notes and draw pictures, ask questions, and fill in as many formal details as you can.
- **Stuck?** Raise your hand; staff will come.
- **At the end:** fill out the reflection survey.